פרק: שימוש בבינה מלאכותית (AI) ככלי עזר לפיתוח ודיבוג חומרה

במהלך פיתוח המערכת, נעשה שימוש אינטנסיבי בבינה מלאכותית (AI) ככלי עזר לתכנון הלוגיקה (RTL) ובעיקר לפתרון תקלות חומרה (Debugging). העבודה מול מודל ה-AI אפשרה לתרגם התנהגות בלתי צפויה של רכיבים פיזיים על גבי לוח ה-DE10-Lite לפתרונות לוגיים בקוד. דוגמה מובהקת לכך התרחשה בעת הוספת אפשרות להפעלת מדידת הפולס באמצעות מתג פיזי (SW[9]).

שלב 1: דרישת המערכת והמימוש הראשוני

המטרה הייתה להרחיב את אפשרויות ההפעלה של מכונת המצבים (FSM). הדרישה שהוגדרה הייתה לאפשר תחילת ספירה לא רק באמצעות לחיצה על הכפתורים הייעודיים (KEY[0] להתחלה, KEY[1] לעצירה), אלא גם על ידי הרמת המתג SW[9] למצב פעיל ('1'). הלוגיקה דרשה שהמערכת תגיב לטריגר הראשון שמופעל – "מי שנלחץ ראשון קובע". הקוד הראשוני שנכתב שילב את SW[9] בצורה ישירה ופשוטה לתוך תנאי המעבר של ה-FSM (פעולת OR לוגית), מתוך הנחה שהרמת המתג שקולה ללחיצת כפתור.

FSM.v
לא ידוע

Top_System12.4.v
לא ידוע

Measurement_Unit.v
לא ידוע

we need to add an option of another measured Pulse length by switching SW[9]

when we get SW[9] = 1, we measure only by it.
so thats mean that we need to duplicate another FSM whos corresponds only to SW[9] and ignores KEY[0:1] inputs.

the first FSM dont act when SW[9] is ON.

maybe we should also duplicate another Counter and to what value to read in the NIOS system

```
84
85      // ============================================================
86      // NEW: SW[9] Edge Detection and Robust MUX Routing (Mode Lock)
87      // MUST BE BEFORE THE FSM!
88      // ============================================================
89      reg sw9_d1, sw9_d2;
90      reg active_mode; // 0 = KEY mode, 1 = SW[9] mode
91
92      always @(posedge MAX10_CLK1_50 or negedge system_reset_n) begin
93          if (!system_reset_n) begin
94              sw9_d1      <= 1'b0;
95              sw9_d2      <= 1'b0;
96              active_mode <= 1'b0;
97          end else begin
98              // Pipeline for edge detection
99              sw9_d1 <= SW[9];
100             sw9_d2 <= sw9_d1;
101
102             // MODE LOCKING:
103             // Latch the state of SW[9] ONLY when the FSM is in IDLE.
104             // w_cnt_reset is exactly 1 when current_state == IDLE.
105             if (w_cnt_reset == 1'b1) begin
106                 active_mode <= SW[9];
107             end
108         end
109     end
110
111     // Detect Rising and Falling edges of SW[9]
112     wire sw9_rise = (sw9_d1 & ~sw9_d2); // Start pulse
113     wire sw9_fall = (~sw9_d1 & sw9_d2); // Stop pulse
114
115     // MUX Logic: Route signals strictly based on the LOCKED active_mode
116     wire final_pulse_start = active_mode ? sw9_rise : w_pulse_start;
117     wire final_pulse_stop  = active_mode ? sw9_fall : w_pulse_stop;
118
119
120     // ============================================================
121     // 2. FSM (Finite State Machine)
122     // ============================================================
123     FSM_Control u_fsm (
```

```
29  );
85  // ======================================================
86     // NEW: SW[9] Debouncing, Edge Detection and Robust MUX
87     // MUST BE BEFORE THE FSM!
88     // ======================================================
89     wire w_sw9_stable;
90     // 1. Debounce SW[9] to prevent bounce-related false stops/starts
91     Debouncer u_deb_sw9 (
92         .clk            (MAX10_CLK1_50),
93         .input_unstable (SW[9]),
94         .output_stable  (w_sw9_stable)
95     );
96     reg sw9_d1, sw9_d2;
97     reg active_mode; // 0 = KEY mode, 1 = SW[9] mode
98
99     always @(posedge MAX10_CLK1_50 or negedge system_reset_n) begin
100        if (!system_reset_n) begin
101            sw9_d1      <= 1'b0;
102            sw9_d2      <= 1'b0;
103            active_mode <= 1'b0;
104        end else begin
105            // 2. Pipeline for edge detection using the DEBOUNCED signal
106            sw9_d1 <= w_sw9_stable;
107            sw9_d2 <= sw9_d1;
108
109            // 3. MODE LOCKING:
110            // Lock the mode ONLY when the FSM is in IDLE (w_cnt_reset == 1)
111            // based on WHICH signal actually triggered the start.
112            if (w_cnt_reset == 1'b1) begin
113                if (sw9_d1 & ~sw9_d2) begin       // If SW[9] rising edge detected
114                    active_mode <= 1'b1;
115                end else if (w_pulse_start) begin // If KEY[0] press detected
116                    active_mode <= 1'b0;
117                end
118            end
119        end
120    end
121    // Detect Rising and Falling edges of the debounced SW[9]
122    wire sw9_rise = (sw9_d1 & ~sw9_d2); // Start pulse
123    wire sw9_fall = (~sw9_d1 & sw9_d2); // Stop pulse
124    // 4. MUX Logic:
125    // Any valid start pulse can trigger the FSM
126    wire final_pulse_start = sw9_rise | w_pulse_start;
127    // Stop pulse is strictly routed based on the locked active_mode
128    wire final_pulse_stop  = active_mode ? sw9_fall : w_pulse_stop;
```

שלב 2: בדיקה בחומרה וגילוי התקלה

לאחר צריבת התצורה אל ה-FPGA ובדיקת המערכת הפיזית, התגלתה תקלה קריטית: כאשר מתג SW[9] הועבר למצב גבוה, המערכת החלה את הספירה אך **נעצרה מיד באופן ספונטני**. ה-FSM איבד יציבות וקפץ בין מצב ריצה (RUN) למצב עצירה (STOP) בצורה בלתי נשלטת.

התקלה תוארה למודל ה-AI כדי להבין את הפער שבין הלוגיקה התיאורטית להתנהגות החומרה הפיזית.

We currently initiate the pulse by setting SW[9] = 1.
However, when we turn it off, the FSM does not react and continues counting until KEY[1] is pressed.
For proper functionality, the FSM should respond only when SW[9] changes to 0, and then stop counting.
Please add the required correction.

how do i prevent מקרי קצה
like - if we already meauring by the keys and we turn the SW[9] on

שלב 3: ניתוח הבעיה והבנת המגבלות הפיזיות

תהליך הדיבוג העלה כי הבעיה נובעת משתי תופעות פיסיקליות ולוגיות שלא קיבלו מענה בקוד המקורי:

1. **רעשים מכניים (Bouncing):** מתג פיזי אינו מחליף מצב באופן אידיאלי מ-'0' ל-'1'. רגע המעבר מלווה ברעידות מכניות במגעים, היוצרות רצף מהיר של עליות וירידות מתח. ה-FSM (הפועל בתדר המערכת של MHz50) דגם את הרעש הזה ופירש אותו כרצף פקודות הפעלה ועצירה.
2. **רמה לוגית קבועה (Level) מול פולס רגעי (Edge):** בעוד שלחיצה על KEY מספקת פולס קצר ומשתחררת, מתג SW נשאר קבוע על '1'. מכונת המצבים דגמה את ה-'1' הקבוע בכל מחזור שעון כיחידת טריגר חדשה באופן רציף, מה שגרם למעברי מצבים בלתי נשלטים.

שלב 4: יישום סינון הרעשים בתוך קובץ ה-Top System

כדי לפתור את הבעיה בצורה יעילה, ה-AI הציע להטמיע מנגנון סינון וזיהוי שפה **ישירות בתוך קובץ ה-**Top_System, לפני שהאות מגיע ל-FSM.

הפתרון כלל שתי שכבות לוגיות:

- **Level Debouncer:** קטע קוד המוודא שהאות המגיע מ-SW[9] נשאר יציב לאורך זמן מסוים (מספר מחזורי שעון) לפני שהוא מאושר כאות תקין, ובכך מסנן את הרעידות המכניות.
- **Edge Detector (זיהוי שפה עולה):** לוגיקה הדוגמת את האות המסונן, ומייצרת פולס הפעלה באורך של **מחזור שעון אחד בלבד** (האות w_SW9_pulse) בעת המעבר מ-'0' ל-'1'.

```
// ========================================================
    // NEW: SW[9] Custom Level-Debouncer & Robust MUX
    // MUST BE BEFORE THE FSM!
    // ========================================================
    reg [15:0] sw9_debounce_cnt;
    reg sw9_stable_level;

    // 1. Custom Level-Debouncer for SW[9]
    // (Keeps the signal continuous, ignores noise, no 1-cycle drops)
    always @(posedge MAX10_CLK1_50 or negedge system_reset_n) begin
        if (!system_reset_n) begin
            sw9_debounce_cnt <= 16'd0;
            sw9_stable_level <= 1'b0;
        end else begin
            if (SW[9] == sw9_stable_level) begin
                sw9_debounce_cnt <= 16'd0; // Reset counter if no change
            end else begin
                sw9_debounce_cnt <= sw9_debounce_cnt + 1'b1;
                if (sw9_debounce_cnt >= 16'd50000) begin // ~1ms stable time
                    sw9_stable_level <= SW[9];
                    sw9_debounce_cnt <= 16'd0;
                end
            end
        end
    end
```

```
109
110     // 2. Edge Detection & Mode Locking
111     reg sw9_d1, sw9_d2;
112     reg active_mode; // 0 = KEY mode, 1 = SW[9] mode
113
114     always @(posedge MAX10_CLK1_50 or negedge system_reset_n) begin
115         if (!system_reset_n) begin
116             sw9_d1      <= 1'b0;
117             sw9_d2      <= 1'b0;
118             active_mode <= 1'b0;
119         end else begin
120             // Pipeline for edge detection using the STABLE LEVEL
121             sw9_d1 <= sw9_stable_level;
122             sw9_d2 <= sw9_d1;
123
124             // 3. MODE LOCKING
125             // Lock the mode ONLY when the FSM is in IDLE (w_cnt_reset == 1)
126             if (w_cnt_reset == 1'b1) begin
127                 if (sw9_d1 & ~sw9_d2) begin       // If SW[9] rising edge detected
128                     active_mode <= 1'b1;
129                 end else if (w_pulse_start) begin // If KEY[0] press detected
130                     active_mode <= 1'b0;
131                 end
132             end
133         end
134     end
135
136     // Detect Rising and Falling edges of the stable level
137     wire sw9_rise = (sw9_d1 & ~sw9_d2); // Start pulse
138     wire sw9_fall = (~sw9_d1 & sw9_d2); // Stop pulse
139
140     // 4. MUX Logic
141     wire final_pulse_start = sw9_rise | w_pulse_start;
142     wire final_pulse_stop  = active_mode ? sw9_fall : w_pulse_stop;
143
144     // ========================================================
145     // 2. FSM (Finite State Machine)
146     // ========================================================
147     FSM_Control u_fsm (
```

שלב 5: מנגנון נעילת המקור הלוגי ב-FSM ("מי שנלחץ ראשון קובע")

מעבר לסינון הרעשים, עלה האתגר התכנוני של בחירת מקור ההפעלה: מה קורה אם המשתמש משחק גם בכפתורים וגם במתג באותו הזמן? כיצד ה-FSM יודע למי מהם להקשיב, ואיך מונעים הפרעות תוך כדי המדידה?

בעזרת ה-AI תוכננה לוגיקה של **"הפרדת נתיבים ונעילה" (State Locking)**:

- **במצב המתנה (IDLE):** מכונת המצבים מאזינה לשני מקורות במקביל – גם ללחיצה על KEY[0] וגם לפולס הנקי w_SW9_pulse. המקור הראשון שמתקבל הוא זה שמוציא את המערכת ממצב המתנה.
- **הנעילה וההתעלמות:** ברגע שאחד מהטריגרים התקבל, ה-FSM עובר למצב ריצה (RUN) תוך כדי הדלקת "דגל" (Flag) או מעבר למצב ספציפי ששומר בזיכרון **מי היה מקור ההפעלה**.
- מרגע זה, כשהמערכת מודדת את הפולס, הלוגיקה **מנתקת מגע (מתעלמת לחלוטין)** מהכניסות האחרות. אם ההפעלה בוצעה על ידי SW[9], לחיצות על KEY[0] או KEY[1] ייפלו על "אוזניים ערלות" ולא ישפיעו כלל על המדידה. ה-FSM ימתין אך ורק לתנאי העצירה התואם של מקור ההפעלה המקורי שלו (למשל, זיהוי שפה יורדת של SW[9]).

סיכום תהליך הדיבוג

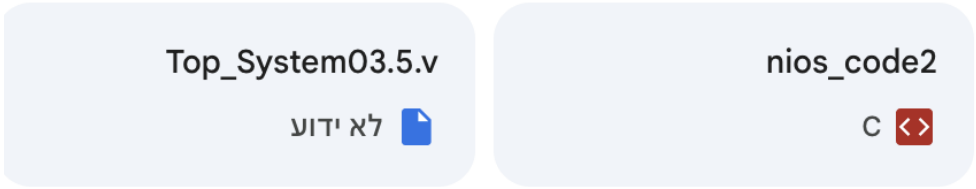
המקרה ממחיש כיצד דיאלוג עם כלי AI מאפשר להעמיק את איכות התכנון. לא רק שנפתרו בעיות פיזיקליות של חומרה (Bouncing ו-Level vs Edge), אלא גם שופרה הארכיטקטורה הלוגית של המערכת כך שתהיה חסינה לטעויות משתמש באמצעות מנגנון בקרת כניסות מוקשח וחסין הפרעות.

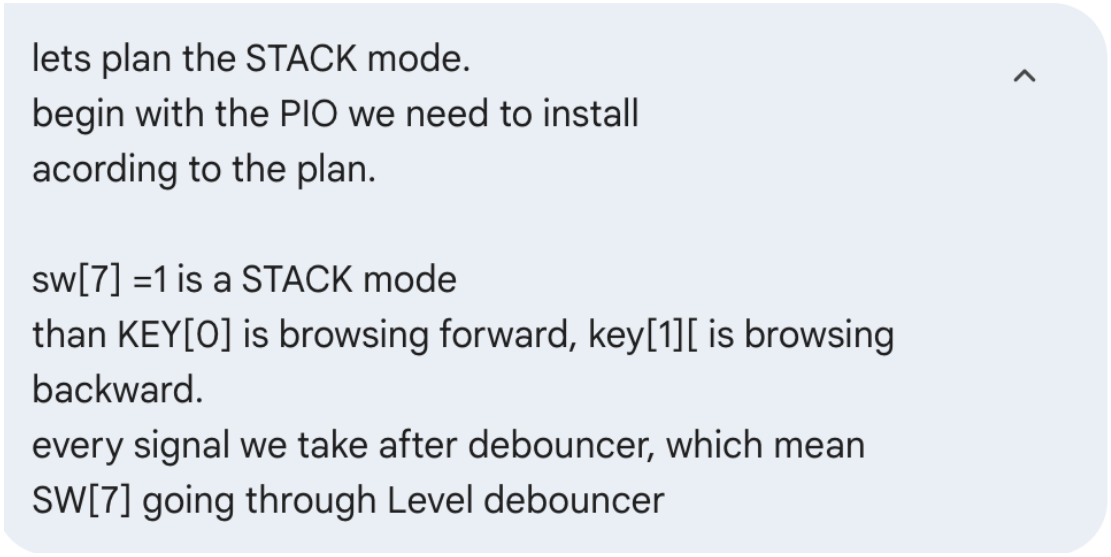
פרק: פתרון בעיית קריאה חזרתית (Polling) בעת שילוב מסך LCD

במהלך הפרויקט, רצינו לשקף למשתמש את מצב המערכת על גבי מסך LCD חיצוני. האתגר המרכזי בשלב זה היה התמודדות עם אופן הקריאה של מתג SW[7] (מצב ה-STACK) על ידי מעבד ה-Nios II, אשר גרם לקריסת התקשורת מול המסך.

שלב 1: הדרישה הראשונית מה-AI

התהליך החל בבקשה ממודל ה-AI לייצר את תשתית הקוד הנדרשת כדי שהמסך יציג את הסטטוס "STACK Mode ON" כאשר מתג SW[7] מורם, ומידע אחר כאשר הוא מורד.

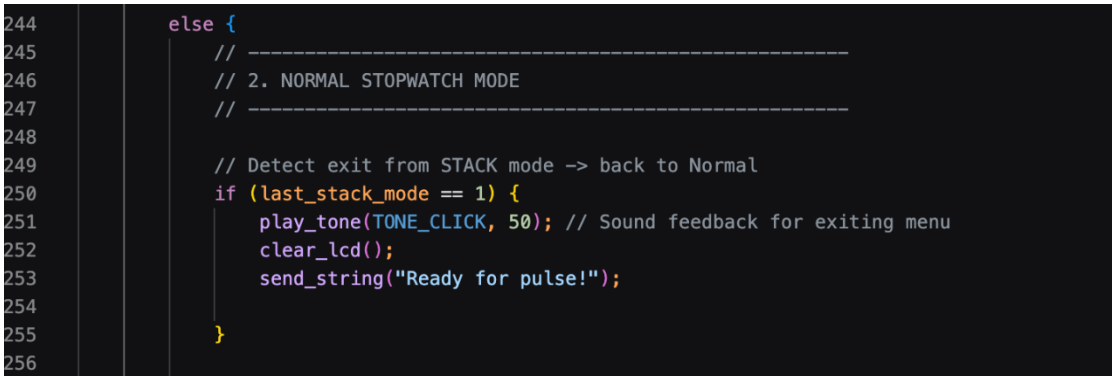




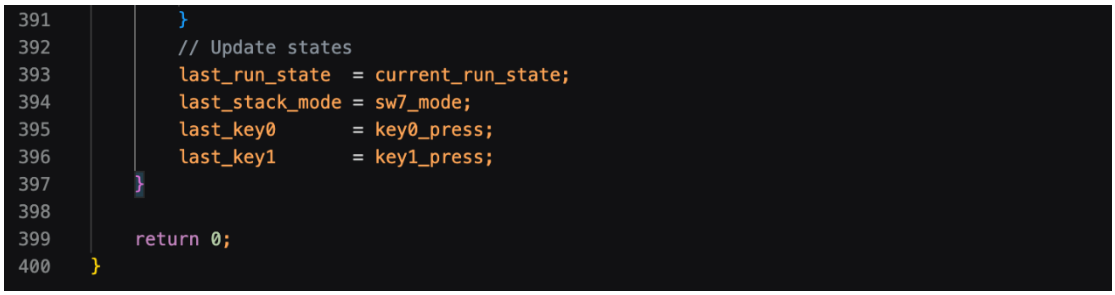
שלב 2: הכנה חומרתית זריזה (ללא התעכבות על תוכנה)

כדי לאפשר קריאה וכתיבה, הוגדרו במהירות ערוצי תקשורת (PIO) ב- Platform Designer, שחיברו בין המעבד לבין פין ה-Busy וקוויי הנתונים של ה-LCD. לאחר מכן הם חווטו בקובץ ה-Verilog הראשי.

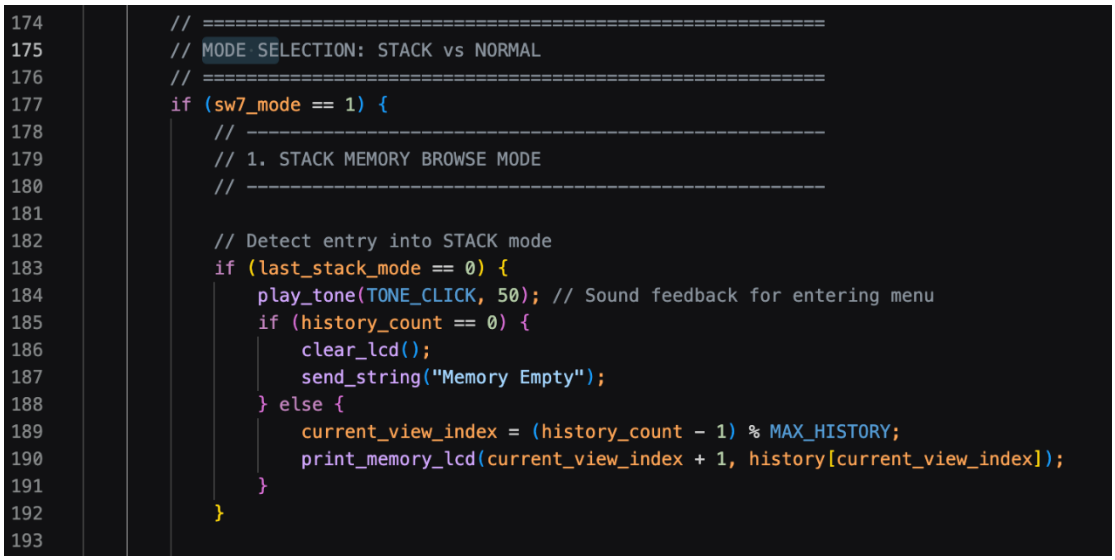
```
144     // State tracking variables
145     int last_run_state = IORD_ALTERA_AVALON_PIO_DATA(PIO_RUN_BASE);
146     int last_stack_mode = 0;
```



```
244         else {
245             // ------------------------------------------------------
246             // 2. NORMAL STOPWATCH MODE
247             // ------------------------------------------------------
248
249             // Detect exit from STACK mode -> back to Normal
250             if (last_stack_mode == 1) {
251                 play_tone(TONE_CLICK, 50); // Sound feedback for exiting menu
252                 clear_lcd();
253                 send_string("Ready for pulse!");
254
255             }
256
```



```
391         }
392         // Update states
393         last_run_state  = current_run_state;
394         last_stack_mode = sw7_mode;
395         last_key0       = key0_press;
396         last_key1       = key1_press;
397     }
398
399     return 0;
400 }
```



```
174         // ======================================================
175         // MODE SELECTION: STACK vs NORMAL
176         // ======================================================
177         if (sw7_mode == 1) {
178             // ------------------------------------------------------
179             // 1. STACK MEMORY BROWSE MODE
180             // ------------------------------------------------------
181
182             // Detect entry into STACK mode
183             if (last_stack_mode == 0) {
184                 play_tone(TONE_CLICK, 50); // Sound feedback for entering menu
185                 if (history_count == 0) {
186                     clear_lcd();
187                     send_string("Memory Empty");
188                 } else {
189                     current_view_index = (history_count - 1) % MAX_HISTORY;
190                     print_memory_lcd(current_view_index + 1, history[current_view_index]);
191                 }
192             }
193
```

שלב 3: בעיית הקריאה של SW[7] (הצפת ה-LCD)

לאחר החיבור, התגלתה תקלה חמורה: המעבד קרא את המצב של SW[7] בתוך לולאת ה-while(1) המהירה. מכיוון שהמתג נשאר למעלה זמן רב (Level), המעבד זיהה בכל איטרציה שצריך לכתוב למסך, ושלח אלפי פקודות הדפסה בשנייה.

בעזרת ה-AI הבנו שהבעיה מתנגשת עם פונקציית שידור התווים שלנו. הפונקציה נכתבה כך שתמתין (Polling) עד שדגל ה-Busy של ה-LCD יירד לפני שליחת התו הבא. ההצפה מקריאת היתר של SW[7] גרמה לפונקציה הזו "להיתקע" בהמתנה אינסופית וליצור עומס שהקריס את המערכת.

```
167  // ======================================================
168   // MUX Logic with STACK Mode Isolation
169   // ======================================================
170
171   // When STACK mode is active (sw7_stable_level == 1), we "mask" the keys
172   // so they don't reach the FSM.
173   wire fsm_key_start = w_pulse_start & ~sw7_stable_level;
174   wire fsm_key_stop  = w_pulse_stop  & ~sw7_stable_level;
175
176   // Start/Stop signals for the FSM
177   wire final_pulse_start = sw9_rise | fsm_key_start;
178   wire final_pulse_stop  = active_mode ? sw9_fall : fsm_key_stop;
179
```

שלב 4: הפתרון – קריאה לפי שינוי מצב בלבד

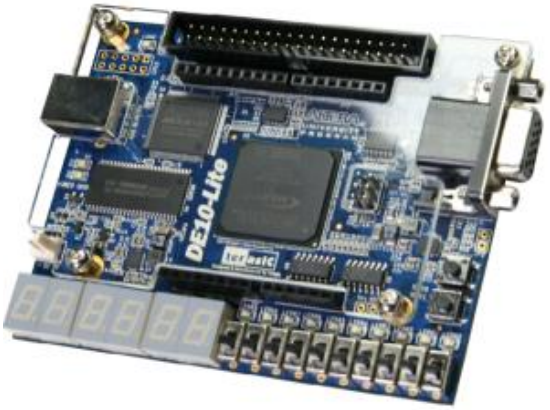
כדי לפתור את התקלה, קוד ה-C תוקן כך שיגיב אך ורק ל**שינוי** במצב של SW[7] (מעבר מ-0 ל-1 ולהפך), ולא למצבו הקבוע. רק בעת זיהוי שינוי, המערכת מנקה את המסך ומפעילה את פונקציות העיצוב (כמו פונקציית הדפסת ההיסטוריה). פתרון זה שחרר את עומס הנתונים ואפשר לממשק לעבוד בצורה חלקה ויציבה.

```
180    // Bundle signals for the NIOS PIO (3 bits)
181    // Bit 0: Mode (SW[7]), Bit 1: Forward (KEY[0]), Bit 2: Backward (KEY[1])
182    wire [2:0] w_stack_control = {~KEY[1], ~KEY[0], sw7_stable_level};
183
```

אינטגרציית המסך עם הלוח

שלב זה, אשר בתכנון המקורי יועד להתבצע במועד מאוחר יותר כחלק מפרק תתי-האינטגרציות של המערכת השלמה, הוקדם והוגדר כיעד מחייב לקראת מצגת האמצע על ידי מנהלת הפרויקטים. פיתוח יכולת זו התברר כאחד האתגרים המורכבים ביותר במהלך חיי הפרויקט, ודרש השקעה חריגה של משאבי זמן, מחקר ותהליכי איתור שגיאות (Debugging) אינטנסיביים.

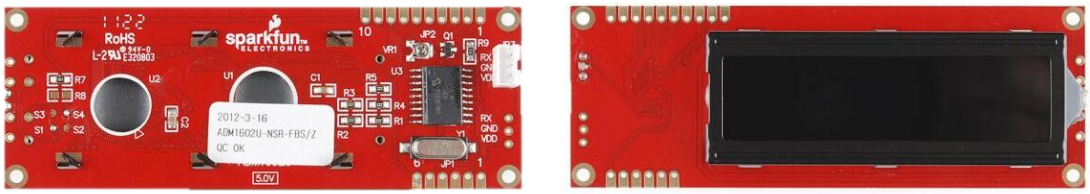
שורש המורכבות נבע מאילוצי החומרה הבסיסיים של הפרויקט: לוח הפיתוח עליו עבדנו, מדגם DE10-Lite, הינו לוח בסיסי אשר אינו כולל בתוכו מסך תצוגה אינטגרלי או מעבד חומרתי מובנה. בשונה מסביבות פיתוח אחרות המגיעות כיחידה אחת הכוללת מסך ומעבד, העבודה עם ה-DE10-Lite דרשה מאיתנו לבנות את כלל תשתיות התקשורת מאפס, החל מהגדרות החיווט הפיזי, דרך פיתוח מנהלי התקנים (Drivers) בחומרה, ועד להטמעת מעבד NIOS II אל תוך רכיב ה-FPGA ויצירת הסנכרון ביניהם.



לוח פיתוח DE10-Lite

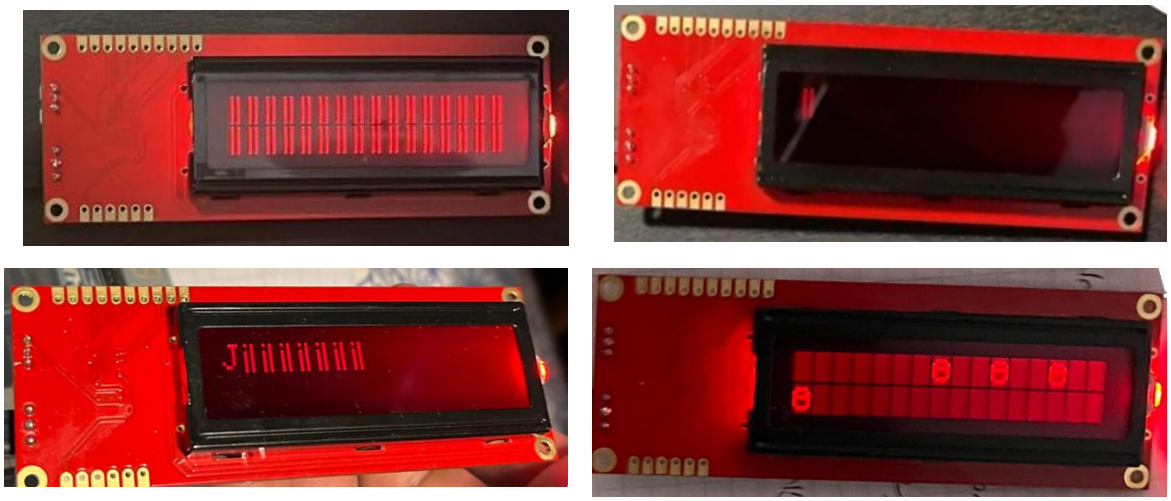
אתגרי החומרה והתקשורת הראשוניים

כצעד ראשון להשגת היעד, שילבנו מערכת מסך תצוגה טורי חיצוני מדגם SerLCD v2.5. חיבור המסך התבצע תוך הסתמכות על מפרט היצרן (Datasheet) והיוועצות עם כלי בינה מלאכותית להגדרת הקצאת הפינים (Pin Planner). עם חיבור המערכת לחשמל, המסך אכן הציג חיווי הפעלה, אך כלל הניסיונות הראשוניים לשדר אליו נתונים באמצעות לוגיקה עוקבת נתקלו בחוסר תגובה מוחלט מצד רכיב התצוגה.



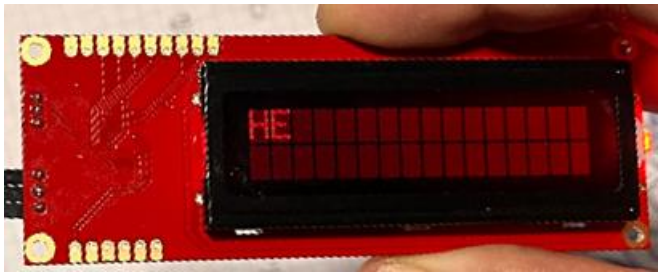
מסך SerLCD v2.5

לאחר ניתוח מעמיק של אפיקי התקשורת, הסקנו כי קיים חוסר תיאום בפרוטוקול העברת הנתונים. ביצענו שינוי משמעותי בתצורת התקשורת והמרנו את קצב השידור (Baud Rate) מ-115,200 ל-9,600 bps . שינוי זה אכן הוליד תגובה ראשונית מהמסך, אך הנתונים שהוצגו היו משובשים לחלוטין וכללו תווים לא מזוהים – תופעה המוכרת כ"ג'יבריש". תופעה זו מעידה לרוב על בעיות סנכרון שעונים, רעשים בקו או חוסר תיאום ברמות המתח הלוגי.



שגיאות תצוגה במסך: הופעת תווים משובשים וטקסט לא קריא ("ג'יבריש").

בשלב זה החלה סדרת בחינות מחמירה. ניסינו לשנות הגדרות מתח ל-$3.3V$, לאמת מחדש את תצורת הפינים בתוכנת ה-Quartus ואף להחליף את החיווט הפיזי מתוך חשש לחוטים בלויים או מגעים רופפים. כל שינוי טופולוגי שכזה הוליד סוגים שונים של שגיאות תצוגה, עד שלבסוף, לאחר כיול מדויק של תזמוני השידור, הצלחנו לקבל לראשונה פלט קריא – שתי האותיות הראשונות של המילה ששידרנו "HE" מתוך המילה (HELLO) .



הצגת חלקה הראשון של המילה הראשונה על גבי המסך.

הצלחה נקודתית זו היוותה את נקודת המפנה שאפשרה לנו להבין את מחזור הפעולה של המסך . בנוסף, הבנו כי תקשורת יציבה דורשת ניהול קפדני של פקודות הבקרה הפנימיות של המסך, המפורטות בטבלת יצרן ייעודית (טבלה 1) .טבלה זו אפשרה לנו לשלב פקודות מערכת הכרחיות, כגון ניקוי התצוגה והזזת הסמן, שהיו קריטיות להצגת טקסט מתחלף בצורה חלקה.

סט פקודות הבקרה והשליטה של מסך התצוגה

Control Character	Command	Description
0xFE	0x01	Clear Display
	0x14	Move Cursor Right 1 Space
	0x10	Move Cursor Left 1 Space
	0x1C	Scroll Right 1 Space
	0x18	Scroll Left 1 Space
	0x0C	Turn Display On / Hide Cursor
	0x08	Turn Display Off
	0x0E	Underline Cursor On
	0x0D	Blinking Cursor On
	0x80 + n	Set Cursor Position
0x7C	0x03	20 Characters Wide
	0x04	16 Characters Wide
	0x05	4 Lines
	0x06	2 Lines
	0x0A	Set Splash Screen
	0x0B	Set Baud Rate to 2400
	0x0C	Set Baud Rate to 4800
	0x0D	Set Baud Rate to 9600
	0x0E	Set Baud Rate to 14400
	0x0F	Set Baud Rate to 19200
	0x10	Set Baud Rate to 38400
	0x80	Backlight Off*
	0x9D	Backlight Fully On*
0x12	Reset to 9600 Baud - send during boot-up	

מימוש מנהל התקן חומרתי מבוסס FSM

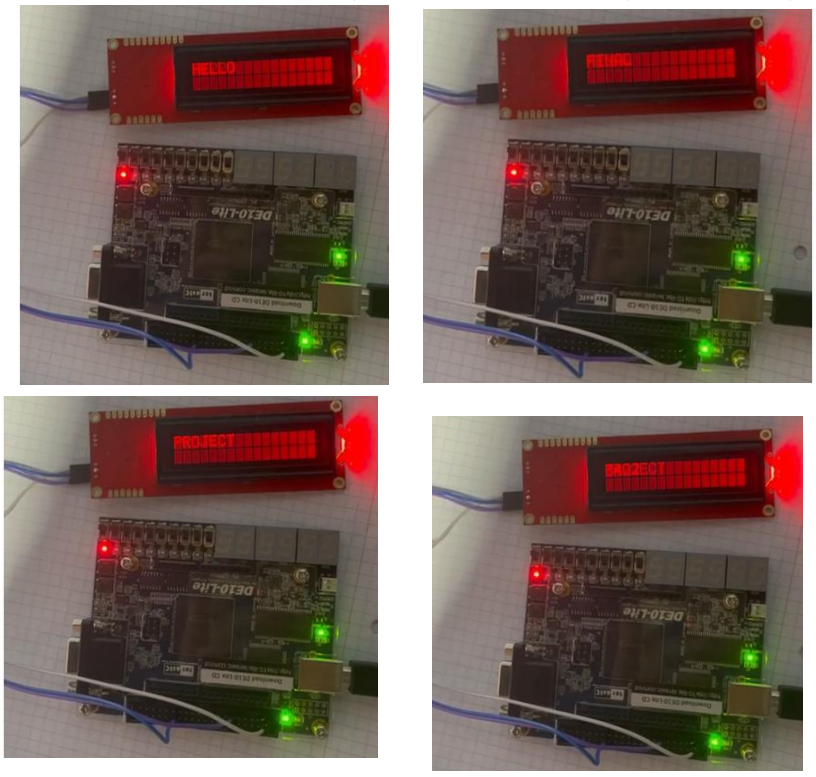
על מנת להוכיח שליטה מלאה על התצוגה, פיתחנו בשלב הראשון מודול חומרתי (מבוסס Verilog) המנהל את התקשורת למסך באופן עצמאי לחלוטין, ללא מעורבות מעבד .ליבת המודול הייתה מכונת מצבים סדרתית (FSM) מורכבת אשר ניהלה את פרוטוקול התקשורת הטורית

(UART). האתגר המרכזי כאן היה תזמון: נדרשנו להמיר את שעון המערכת המהיר (50MHz) לקצב שידור איטי ויציב של Baud 9600 באמצעות חלוקת תדר מדויקת.

מכונת המצבים תוכננה לבצע רצף פעולות לוגי קפדני ומוגדר מראש:

1. **אתחול וייצוב:** שידור רצף של פקודות איפוס ראשוניות עם עליית המתח, כדי להבטיח שהמסך יוצא ממצבי שגיאה ומתייצב על קצב התקשורת הרצוי.
2. **מחזור ניקוי:** שליחת פקודת בקרת תצוגה לניקוי המסך, מלווה בהשהיה חומרתית יזומה (המתנה של אלפי מחזורי שעון) הנדרשת לבקר ה-LCD הפנימי כדי להשלים את מחיקת הפיקסלים.
3. **שידור נתונים אסינכרוני:** כניסה ללולאת שידור שבה מועברת מחרוזת תווים קבועה מראש (כגון "HELLO" או "PROJECT"). מכונת המצבים דאגה לפרק כל תו לסיביות, לעטוף אותו בסיביות התחלה ועצירה, (Start/Stop bits) ולשדרו תוך שמירה על מרווחי זמן (Byte Gap) בין תו לתו למניעת הצפת חוצץ (Buffer Overflow) במסך.

מודול זה פעל בהצלחה והציג את המילים על המסך בצורה מחזורית, מה שהוכיח הלכה למעשה כי בעיות התקשורת והתזמון הפיזיות נפתרו לחלוטין, וכי החומרה שפיתחנו יציבה.



הצגת הודעות ללא שימוש במעבד

אינטגרציה עם מערכת העיבוד (NIOS II)

אף על פי שהוכחנו יכולת שליטה חומרתית מלאה, היעד ההנדסי הסופי, כפי שהוגדר מראש, היה לנהל את התצוגה והודעות הטקסט דרך מעבד ה-NIOS II. גישה מערכתית זו הכרחית כדי

לאפשר למערכת לקבל בהמשך הפרויקט נתונים משתנים (כגון זמן המדידה של הפולס) ולהציגם באופן דינמי, יכולת שכמעט בלתי אפשרי ליישם ביעילות בחומרה טהורה.

לשם כך, ביצענו תהליך של "פירוק והרכבה" ארכיטקטוני. לקחנו את מנהל ההתקן החומרתי שיצרנו בשלב הקודם, והסרנו ממנו את הלוגיקה החכמה (מילון המילים ומכונת המצבים הגבוהה). המודול החומרתי צומצם לכדי רכיב קצה תפעולי בלבד, המקושר לאפיקי הקלט/פלט (PIO) של עורק הנתונים במערכת ה-Qsys.

שילוב של מעבד פנימי (Soft-Core Processor) בתוך חומרת FPGA מותאמת אישית הוא ציון דרך משמעותי בכל פרויקט אמבדד. זהו תהליך שדורש "תפירה" מדויקת בין עולם תכנון החומרה לעולם הנדסת התוכנה. להלן סקירה מקיפה ושלב-אחר-שלב של תהליך ההתקנה והאינטגרציה המלא, מהקמת המעבד ב-Quartus ועד הרצת קוד ה-C בסביבת ה-Eclipse.

שלב 1: בניית חומרת המעבד ב Platform Designer (Qsys)-השלב הראשון מתבצע כולו בתוך תוכנת Quartus, ומטרתו היא להרכיב את תשתית החומרה של המעבד שלנו, להקצות לו זיכרון, וליצור את "הגשרים" (PIOs) בינו לבין שאר המערכת.

1. **פתיחת הסביבה:** מתוך Quartus, פתחנו את ה-Platform Designer (לשעבר Qsys) ופתחנו מערכת חדשה.
2. **הוספת המעבד (Nios II Processor):** הוספנו את ה-IP של Nios II (בתצורת e/ החסכונית או f/ המהירה). זהו ה"מוח" של המערכת שיריץ את הפקודות.
3. **הוספת זיכרון (On-Chip Memory):** הוספנו רכיב RAM פנימי לאחסון קוד ה-C שייכתב בהמשך (Instruction Memory) ואת המשתנים (Data Memory). חיברנו את קווי ה-Instruction וה-Data Master של ה-Nios לזיכרון זה.
4. **הוספת JTAG UART:** רכיב קריטי המאפשר לתקשר עם המעבד מהמחשב דרך כבל ה-USB, מה שמאפשר הדפסות דיבאג (כמו printf) ישירות לקונסולה.
5. **הוספת PIOs (Parallel I/O):** כאן נוצר החיבור למערכת ה Verilog-שלנו:
   - **כניסות ל-Nios (Inputs) :** הוספנו PIOs שמוגדרים כ-Input עבור נתונים מהחומרה, למשל shadow_out (לקבלת המדידה) ו-nios_ready (קבלת דגל מוכנות).
   - **יציאות מה-Nios (Outputs):** הוספנו PIOs שמוגדרים כ Output-עבור פקודות שמערכת ה C-שולחת החוצה, למשל uart_data) uart_write-לשליחת נתונים למסך ה.(LCD-
6. **חיווט (Routing) והגדרות ב Qsys:-**חיברנו את שעון המערכת (clk) וה ,Reset-ואת ה-Data Masterשל ה Nios-אל כניסות ה Slave-של כל ה) PIOs-על גבי ערוץ ה .(Avalon-בעמודת ה ,Export-נתנו שם לכל הדק שרצינו לחבר ב.Verilog-

7. **הקצאת כתובות וקימפול:** ביצענו Assign Base Addresses כדי שלכל PIO תהיה כתובת זיכרון ייחודית ב-C, ולחצנו על Generate HDL לייצור קובץ ה-Verilog וקובץ ה-sopcinfo הקריטי.

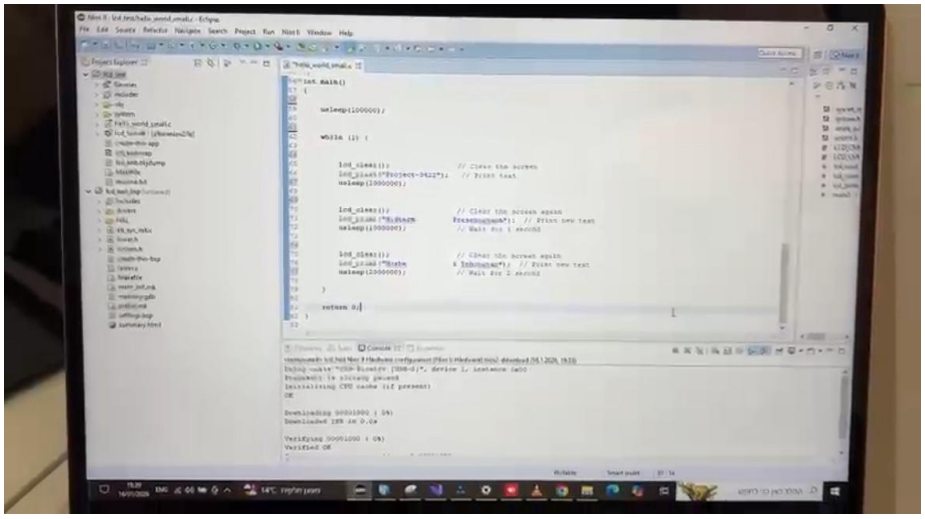
שלב 2: שילוב ה-Nios במערכת ה-Verilog וצריבת החומרה אחרי שהמעבד נוצר כבלוק לוגי, הכנסנו אותו לפרויקט הקיים לקראת צריבה ללוח ה-DE10-Lite:

1. **Instantiation ב-Top_System.v:** פתחנו את המודול הראשי והוספנו קריאה למופע של מערכת ה-Qsys (למשל nios_system u_nios).
2. **חיבור החוטים (Port Mapping):** חיברנו את הפורטים שיצאו מה-Nios אל ה-Wires הרלוונטיים במערכת (לדוגמה, החוט הפנימי w_shadow_out שיצא מיחידת המדידה חובר לכניסת shadow_out_export).
3. **קימפול וצריבה:** הרצנו קימפול Quartus מלא כדי למפות את הלוגיקה לשערים הפיזיים של ה-FPGA. לאחר מכן, צרבנו את קובץ החומרה (.sof) ללוח באמצעות ה-Programmer.

שלב 3: כתיבת התוכנה והרצתה ב-Nios II SBT for Eclipse בשלב זה, את כלל הלוגיקה החכמה שהוסרה מקוד ה-Verilog העברנו אל עולם התוכנה, באמצעות קוד C שרץ על המעבד.

1. **יצירת פרויקט ו-BSP:** פתחנו את סביבת Eclipse, יצרנו פרויקט מבוסס טמפלייט והזנו את קובץ ה-sopcinfo.. הקובץ יצר את ה-BSP (Board Support Package) – ספריה המכילה את כל הדרייברים והמיפויים של כתובות הזיכרון הספציפיות של ה-PIOs שלנו.
2. **לוגיקת העבודה) קוד ה-C):** התבססנו על ספריות המערכת (system.h ו-altera_avalon_pio_regs.h) לקריאה וכתיבה ישירה מהחומרה. (IORD / IOWR) בסביבת התוכנה, בנינו מנגנון עבודה מאובטח הכולל:
   - **פרוטוקול "לחיצת יד" (Handshake):** "טרם שידור כל תו, המעבד מבצע דגימה (Polling) של דגל חומרתי המעיד אם הקו תפוס ("Busy"). המעבד ממתין בלולאה ואינו משדר נתון חדש עד שהחומרה מסיימת לשדר את התו הקודם.
   - **ניהול שידור אקטיבי:** ברגע שהקו פנוי, התוכנה טוענת את הערך לאוגר הנתונים (uart_data), ומחוללת פולס דיגיטלי קצר דרך אוגר PIO נפרד (uart_write), המשמש כהדק (Trigger) לפעולת השידור בחומרה.
   - **בקרת זרימה והשהיות תוכנתיות:** כתיבת פונקציות עיליות לניהול מחרוזות ארוכות ולשליחת פקודות בקרה (כמו ניקוי מסך), תוך שילוב פקודות השהיה (Sleep) מתוכנתות המותאמות בדיוק למגבלות האלקטרוניות של רכיב ה-LCD.
3. **קימפול והרצה:** ביצענו קימפול לתוכנה ליצירת קובץ ה-elf.. לחיצה על "Run As -> Nios II Hardware" שלחה את הקובץ דרך כבל ה-JTAG לזיכרון ה-FPGA. המעבד אותחל, התחיל להריץ את הקוד, והשלים את מעגל העבודה בין התוכנה לחומרה.

בזכות אינטגרציה מערכתית זו, המעבד הריץ בהצלחה את התוכנית הראשית, וניהל תצוגה דינמית של מסכי הפתיחה למצגת האמצע, "Midterm Presentation" שמות חברי הצוות, ועוד (בזה אחר זה, בתזמון מושלם וללא תקלות שידור.



הטמעת הקוד במעבד

הצגת הודעות טקסט עם שימוש במעבד